

CIS Benchmarks

[Autopilot](#) (/kubernetes-engine/docs/concepts/autopilot-overview)

[Standard](#) (/kubernetes-engine/docs/concepts/choose-cluster-mode)

This page describes the approach that Google Kubernetes Engine (GKE) takes to improve compliance with the [Center for Internet Security \(CIS\) benchmarks](#) (<https://www.cisecurity.org/cis-benchmarks>) for Kubernetes and for GKE. This page includes the following information:

- How we configure the managed GKE control plane to conform to the CIS Kubernetes Benchmark
- How you can configure your GKE nodes and workloads to conform to the CIS Google Kubernetes Engine (GKE) Benchmark

About the CIS Benchmarks

CIS releases the following benchmarks that contain secure configuration guidelines for Kubernetes:

- **CIS Kubernetes Benchmark:** Applies to the open source Kubernetes project. Intended to provide guidance for a variety of self-managed and hosted Kubernetes implementations.
- **CIS GKE Benchmark:** Establishes guidelines for the secure configuration of components you can control in GKE clusters. Includes recommendations that are specific to GKE on Google Cloud.

We recommend that you **prioritize the CIS GKE Benchmark**, because it is specific to GKE on Google Cloud. The CIS Kubernetes Benchmark contains many recommendations for controls that you can't view or modify in GKE. Our approach to cluster security includes mitigations that go beyond the scope of the open source Kubernetes benchmark and might result in conflicts with those recommendations.

Other benchmarks that apply to GKE

In addition to the CIS GKE Benchmark and the CIS Kubernetes Benchmark, the following benchmarks apply to the operating systems that are available in GKE. Even if a specific OS benchmark doesn't explicitly address Kubernetes usage, you should still reference that benchmark for additional security guidance.

- **Container-Optimized OS Benchmark** (https://www.cisecurity.org/benchmark/google_cloud_computing_platform): the default operating system that's installed on all GKE Linux nodes
- **Ubuntu Linux Benchmark** (https://www.cisecurity.org/benchmark/ubuntu_linux): available for GKE Standard
- **Windows Server Benchmark** (https://www.cisecurity.org/benchmark/microsoft_windows_server): available for GKE Standard

The default container runtime, containerd, doesn't have a benchmark.

Shared responsibility model

Based on the [GKE shared responsibility model](#)

([/kubernetes-engine/docs/concepts/shared-responsibility](#)), we manage the following components for you:

- The control plane, including the control plane VMs, API server, and components like the cluster state database (etcd or Spanner-based), kube-controller-manager, and kube-scheduler.
- The node operating system.

These components exist in a project that GKE owns, so you can't modify or evaluate any of these components against corresponding CIS Benchmark controls. You can, however, evaluate and remediate any CIS Benchmark controls that apply to your worker nodes and your workloads. Based on the GKE shared responsibility model, these components are your responsibility.

Our approach to securing GKE for the CIS Benchmark

GKE is a managed implementation of open source Kubernetes. We fully manage the control plane and are responsible for securing the configuration of control plane components. The

following table describes some of our decisions that might affect scoring of the CIS benchmarks:

GKE security approach

Authentication	- Some GKE monitoring components use anonymous authentication to obtain metrics. - Some control plane components are bootstrapped using static tokens, which are then used to authenticate to the API server. These tokens are generated every time a VM starts or restarts.
Admission controllers	GKE disables the following admission controllers: EventRateLimit: This is an alpha feature in Kubernetes AlwaysPullImages: This controller provides some protection for private registry images in noncooperative multitenant clusters, at the cost of making image registries a single point of failure for creating new Pods in the cluster. SecurityContextDeny: The Pod Security Admission controller (/kubernetes-engine/docs/how-to/podsecurityadmission) is preferred and is available in all GKE editions. You can also enable enforcement of the Pod Security Standards by using Policy Controller (/kubernetes-engine/policy-controller/docs/overview). ImagePolicyWebhook: GKE disables ImagePolicyWebhook by default because it has its own mechanisms for image management and security. This lets GKE maintain stricter control over the environment and ensure that its security practices are consistently applied. However, you can use Binary Authorization or Policy Controller for policy management.
Audit logging	GKE captures audit logs using the GKE audit policy (/kubernetes-engine/docs/concepts/audit-policy). As a result, we don't need to set any Kubernetes API server audit logging flags.
Debugging	GKE uses profiling for debugging.
Encryption	GKE encrypts customer content at rest by default (/security/encryption/default-encryption). You can optionally use a key that you manage to encrypt your data. For details, see Encrypt secrets at the application layer (/kubernetes-engine/docs/how-to/encrypting-secrets).

GKE security approach

	GKE uses mTLS for traffic between control plane components and between the control plane and nodes. For details, see Cluster trust (/kubernetes-engine/docs/concepts/cluster-trust).
etcd	In open source Kubernetes, the cluster state database uses <i>etcd</i>. In GKE, the backend database that stores the cluster's state is one of the following technologies: etcd: the cluster runs etcd instances on the control plane. Spanner: GKE stores the cluster state in a Spanner-based database that's separate from the control plane VMs. All GKE clusters serve the etcd API in control plane VMs. Any client interactions with Kubernetes API are the same as in open source Kubernetes. Depending on the database technology that's the backend for the etcd API in your cluster, you might notice discrepancies in any etcd-related scoring in the open source CIS Kubernetes Benchmark.
kubelet	- GKE enables the unauthenticated kubelet read-only port. You can disable the port by setting <code>--no-autoprovisioning-enable-insecure-kubelet-readonly-port</code> (/kubernetes-engine/docs/how-to/disable-kubelet-readonly-port). The read-only port will be disabled by default in a future release after allowing you some time to migrate. - GKE Standard mode lets your workloads modify kernel defaults if needed. - GKE limits the number of Kubernetes events in the kubelet to reduce the risk of denial-of-service attacks. - GKE uses mTLS to secure traffic between the kubelet and the API server. - GKE rotates server certificates by default, and rotates client certificates when Shielded GKE Nodes is enabled. - GKE uses the golang default allowed cipher set (https://go.dev/blog/tls-cipher-suites), which is also the default for Kubernetes.

Evaluate GKE against the CIS Benchmarks

Note: This section mentions third-party applications like **kube-bench**. The versions of the CIS Benchmarks that these applications evaluate might not be the latest available versions. Ensure that you check which version your chosen application uses for evaluations.

You can automate evaluation of your clusters against the Benchmarks by using one of the following methods:

- CIS GKE Benchmark:
 - Run **kube-bench** to evaluate worker nodes against the Benchmark. For details, see the [kube-bench GitHub repository](https://github.com/aquasecurity/kube-bench) (<https://github.com/aquasecurity/kube-bench>).
 - Use a third-party tool like Twistlock Defender to evaluate nodes against the Benchmark.
- CIS Kubernetes Benchmark: Run **kube-bench** to evaluate worker nodes against the Benchmark. You can't evaluate the managed control plane against those recommendations in the Benchmark.

What's next

- Read the [GKE security overview](/kubernetes-engine/docs/concepts/security-overview) (/kubernetes-engine/docs/concepts/security-overview).
- Follow security best practices in the [GKE hardening guide](/kubernetes-engine/docs/how-to/hardening-your-cluster) (/kubernetes-engine/docs/how-to/hardening-your-cluster).
- Learn about monitoring your clusters for security issues with [GKE security posture](/kubernetes-engine/docs/concepts/about-security-posture-dashboard) (/kubernetes-engine/docs/concepts/about-security-posture-dashboard).
- Learn how to evaluate your clusters for compliance issues in the [GKE compliance dashboard](/kubernetes-engine/fleet-management/docs/about-compliance-dashboard) (/kubernetes-engine/fleet-management/docs/about-compliance-dashboard).

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-07-10 UTC.